

Adding page transition animations in the Next.js App Router can be achieved using a library like **Framer Motion** or the experimental **View Transition API**. The most common approach uses Framer Motion within a `template.tsx` file for smooth entry animations, or a more complex workaround using `AnimatePresence` and router context manipulation for both entry and exit animations.  YouTube +2

Method 1: Using Framer Motion with `template.tsx`

This method provides simple entry animations that replay on every navigation.

The `template.tsx` file is remounted on navigation, unlike a `layout.tsx`, which makes it suitable for re-triggering entrance animations.  Medium +2

1. Install Framer Motion:

bash

```
npm install framer-motion  
# or yarn add framer-motion  
# or bun add framer-motion
```

2. Create a `template.tsx` file in your root `app` directory (or specific route group where you want the animation to apply) and add the following client component code:

tsx

```
// app/template.tsx  
"use client";  
  
import { motion } from "framer-motion";  
import React from "react";  
  
export default function Template({ children }: { children: React.ReactNode }) {  
  return (  
    <motion.div  
      initial={{ opacity: 0, y: 20 }} // Initial state  
      animate={{ opacity: 1, y: 0 }} // Final state  
      transition={{ type: "spring", stiffness: 260, damping: 20 }} // Transition proper  
    >  
      {children}  
    </motion.div>  
  );  
}
```

 Medium +4

Method 2: Using Framer Motion with Exit Animations (Advanced)

To achieve both entry and *exit* animations, a more involved approach is needed to manage the router's context and prevent the immediate unmounting of the old page content. This typically involves using `AnimatePresence` along with helper components to "freeze" the previous route's content while the exit animation plays. 

A complete code example for this advanced technique, including the necessary `FrozenRouter` and `LayoutTransition` components, is detailed in resources like imcorfitz.com or the [Stack Overflow solution](#). 

Method 3: Using the Experimental View Transition API

Next.js provides experimental support for the native browser **View Transition API**, which allows for seamless animated page transitions with less code, though browser support may vary.  YouTube +1

1. **Enable the experimental flag** in your `next.config.js` file:

javascript 

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    viewTransitions: true,
  },
};

module.exports = nextConfig;
```

2. **Add the necessary CSS** to define the appearance and animation of the default view transition.  YouTube +4

This method is simpler but is considered experimental and not recommended for production use at this time. 